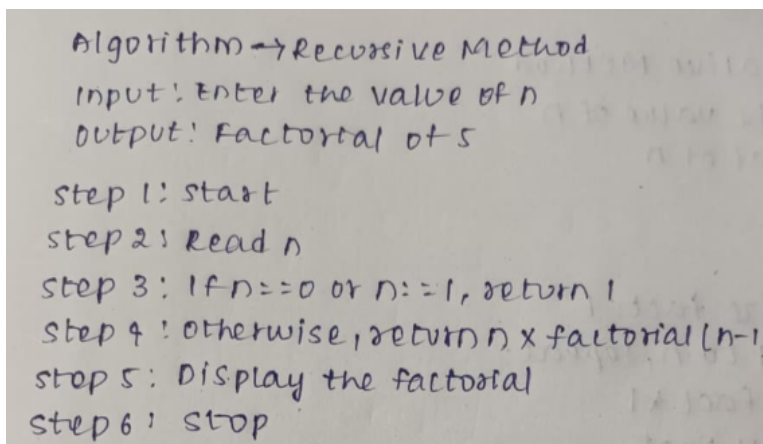


Practical-4

Aim: Implementation and Time analysis of factorial program using iterative and recursive method.

Algorithm:



Algorithm $\rightarrow$ Recursive Method
Input: Enter the value of n
Output: Factorial of 5

step 1: start
step 2: Read n
step 3: If $n == 0$ or $n == 1$, return 1
step 4: otherwise, return $n \times \text{factorial}(n-1)$
step 5: Display the factorial
step 6: stop

Program:

1)Recursive method:

```
#include <iostream>
```

```
using namespace std;
```

```
int factorial(int n)
```

```
{
```

```
    if (n == 0 || n == 1)
```

```
        return 1;
```

```
    else
```

```
        return n * factorial(n - 1);
```

```
}
```

```
int main()
```

Name:Tarmisha

En no:92460118029

```
{
    int n;

    cout << "Enter a number: ";

    cin >> n;

    cout << "Factorial using Recursive Method = " << factorial(n);

    return 0;
}
```

Output:

```
Enter a number: 5
Factorial using Recursive Method = 120

=== Code Execution Successful ===
```

Time complexity:

Time complexity: Recursive Method

```
if (n==0 || n==1) → 1
    return 1; → 1
return n * factorial(n-1); → n
int main() {
    int n; → 1
    cout << "Enter n: "; → 1
    cin >> n; → 1
    cout << "Factorial "
    cin >> factorial(n); → n
    return 0; → 1
}
```

$$T(n) = 1 + 1 + 1 + n + 1 + 1 + (n-1) + 1$$

$$= 3n + 4$$

$$= O(n) - \text{Average Case}$$

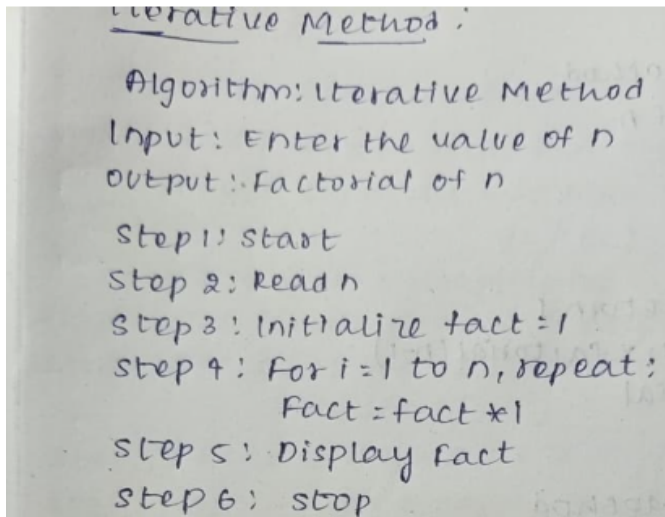
Best case: When $n=0$ (or) $n=1$, the base condition is immediately satisfied. so there is only one recursive call $O(1)$

Average case: For a normal input such as $n=5$, the function makes approximately n recursive calls $O(n)$

Worst case: For a large input the function continues recursive until it reaches $n=1$, resulting in recursive calls $O(n)$

2) Iterative Method:

Algorithm:



Program:

```
#include <iostream>

using namespace std;

int main()
{
    int n, fact = 1;

    cout << "Enter a number: ";
    cin >> n;

    for (int i = 1; i <= n; i++)
    {
        fact = fact * i;
    }
}
```

```
cout << "Factorial using Iterative Method = " << fact;
```

```
return 0;
```

```
}
```

Output:

```
Enter a number: 3
Factorial using Iterative Method = 6
```

```
=== Code Execution Successful |===
```

Time complexity:

Input: Enter the value of n
Output: Factorial of n

Step 1: Start
Step 2: Read n
Step 3: Initialize fact = 1
Step 4: For i = 1 to n, repeat:
 fact = fact * i
Step 5: Display fact
Step 6: Stop

Time complexity: Iterative Method

```
int main() { → 1
    int n; → 1
    long long fact = 1; → 1
    cout << "Enter the value of n: "; → 1
    cin >> n; → 1
    for (int i = 1; i <= n; i++) { → 1 + (n+1) + n
        fact = fact * i; → n
    }
    cout << "Factorial: " << fact; → 1
    return 0; → 1
}
```

$T(n) = 1 + 1 + 1 + 1 + (n+1) + n + n + 1 + 1$
 $T(n) = 3n + 9$

Best case: $T(n) = O(n)$
 Average case: $O(n)$
 In the iterative factorial program the number of iterations not depend on the type of value of the input n, the loop always execute exactly n times.
 Worst case: The loop always executes n times even in the worst case of the iteration $O(n)$



Faculty of Engineering & technology

Department of CSE-AI&ML

Design and analysis of algorithm – 01AI0506

Conclusion:

The factorial program was successfully implemented using both iterative and recursive methods. The iterative method has $O(n)$ time and $O(1)$ space complexity, while the recursive method has $O(n)$ time and $O(n)$ space complexity due to the recursive call stack.